



## Module Guide for Software Engineering

Team 13, Speech Buddies

Mazen Youssef

Rawan Mahdi

Luna Aljammal

Kelvin Yu

April 6, 2026

# 1 Revision History

| Date          | Version | Notes                                                                                       |
|---------------|---------|---------------------------------------------------------------------------------------------|
| Nov 11, 2025  | 1.0     | Added modules and Traceability Matrix                                                       |
| Nov 12, 2025  | 1.1     | Added Modules M6-M11                                                                        |
| Nov 12, 2025  | 1.2     | Added Modules M15-M18                                                                       |
| Nov 26, 2025  | 1.3     | Added Modules M1-M6                                                                         |
| Jan 10, 2026  | 1.4     | Implemented TA Feedback for M7-M11                                                          |
| Jan 10, 2026  | 1.4     | Implemented TA and Peer Feedback for AC/UC distinctions                                     |
| Jan 21, 2026  | 2.0     | Merged modules in application layer and updated timeline                                    |
| Jan 21, 2026  | 2.1     | Added communication protocol details                                                        |
| Feb 10, 2026  | 2.2     | Implemented advisor feedback on structure, clarity and terminology                          |
| Feb 16, 2026  | 2.3     | Updated module descriptions to resemble rev0 architecture                                   |
| Feb 23, 2026  | 2.4     | Implemented advisor feedback on module classification and command refinement                |
| Apr 01, 2026  | 2.5     | Updated M15 module description to match <code>audio_capture.py</code> implementation        |
| Apr 01, 2026  | 2.6     | Updated M17 module description to match <code>whisper_lora_trainer.py</code> implementation |
| April 1, 2026 | 3.0     | Updated module descriptions to reflect final architecture                                   |
| April 5, 2026 | 3.1     | Addressed final TA feedback                                                                 |

*Note: The template's traditional hardware-hiding hierarchy was reviewed during TA feedback resolution. VoiceBridge uses a layered software architecture, so that categorization was not applied directly where it did not map cleanly to the implemented modules.*

## 2 Reference Material

This section records information for easy reference.

### 2.1 Abbreviations and Acronyms

| symbol | description                         |
|--------|-------------------------------------|
| AC     | Anticipated Change                  |
| DAG    | Directed Acyclic Graph              |
| M      | Module                              |
| MG     | Module Guide                        |
| OS     | Operating System                    |
| R      | Requirement                         |
| SC     | Scientific Computing                |
| SRS    | Software Requirements Specification |
| UC     | Unlikely Change                     |
| FR     | Functional Requirement              |
| LF     | Look and Feel                       |
| PF     | Performance                         |
| OER    | Operational & Environmental         |
| MS     | Maintainability & Support           |
| SEC    | Security                            |
| CUL    | Cultural                            |
| CPL    | Compliance                          |
| TLS    | Transport Layer Security            |

# Contents

|           |                                                   |           |
|-----------|---------------------------------------------------|-----------|
| <b>1</b>  | <b>Revision History</b>                           | <b>i</b>  |
| <b>2</b>  | <b>Reference Material</b>                         | <b>ii</b> |
| 2.1       | Abbreviations and Acronyms . . . . .              | ii        |
| <b>3</b>  | <b>Introduction</b>                               | <b>1</b>  |
| <b>4</b>  | <b>Anticipated and Unlikely Changes</b>           | <b>2</b>  |
| 4.1       | Anticipated Changes . . . . .                     | 2         |
| 4.2       | Unlikely Changes . . . . .                        | 3         |
| <b>5</b>  | <b>Module Hierarchy</b>                           | <b>4</b>  |
| 5.1       | Architectural Terminology . . . . .               | 4         |
| 5.2       | Design Rationale . . . . .                        | 5         |
| 5.3       | Module Listing . . . . .                          | 5         |
| <b>6</b>  | <b>Connection Between Requirements and Design</b> | <b>6</b>  |
| <b>7</b>  | <b>Module Decomposition</b>                       | <b>8</b>  |
| <b>8</b>  | <b>Traceability Matrix</b>                        | <b>17</b> |
| <b>9</b>  | <b>Use Hierarchy Between Modules</b>              | <b>19</b> |
| <b>10</b> | <b>User Interfaces</b>                            | <b>20</b> |
| <b>11</b> | <b>Design of Communication Protocols</b>          | <b>24</b> |
| <b>12</b> | <b>Timeline</b>                                   | <b>24</b> |

## List of Tables

|    |                                                                              |    |
|----|------------------------------------------------------------------------------|----|
| 1  | Module Hierarchy . . . . .                                                   | 7  |
| 2  | Trace Between Functional Requirements and Modules . . . . .                  | 17 |
| 3  | Trace Between Look & Feel Requirements and Modules . . . . .                 | 18 |
| 4  | Trace Between Usability & Humanity Requirements and Modules . . . . .        | 18 |
| 5  | Trace Between Performance Requirements and Modules . . . . .                 | 18 |
| 6  | Trace Between Operational & Environmental Requirements and Modules . . . . . | 18 |
| 7  | Trace Between Maintainability & Support Requirements and Modules . . . . .   | 19 |
| 8  | Trace Between Security Requirements and Modules . . . . .                    | 19 |
| 9  | Trace Between Cultural Requirements and Modules . . . . .                    | 19 |
| 10 | Trace Between Compliance Requirements and Modules . . . . .                  | 19 |

|    |                                      |    |
|----|--------------------------------------|----|
| 11 | Module Completion Schedule . . . . . | 25 |
|----|--------------------------------------|----|

## List of Figures

|   |                                                             |    |
|---|-------------------------------------------------------------|----|
| 1 | Use hierarchy among modules . . . . .                       | 20 |
| 2 | VoiceBridge browser toolbar icon . . . . .                  | 20 |
| 3 | Browser extension widget toggle interface . . . . .         | 21 |
| 4 | Main VoiceBridge application interface . . . . .            | 21 |
| 5 | Live transcription output panel . . . . .                   | 22 |
| 6 | Speech input interface while listening . . . . .            | 22 |
| 7 | Detected voice command confirmation screen . . . . .        | 23 |
| 8 | Confirmation screen with command refinement input . . . . . | 23 |

### 3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). For VoiceBridge, we adopt this decomposition based fundamentally on Parnas’s principle of information hiding (Parnas, 1972). This principle supports design for change, because the “secrets” that each module hides represent likely future changes. This approach supports design for change, a critical concern in assistive technologies where adaptation to evolving user needs and cutting-edge speech recognition improvements frequently occur, especially in early development phases.

Our design follows the rules layed out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module’s data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers’ understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness

of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules.

## 4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

### 4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

**AC1:** Consumer-grade hardware platform and input devices.

VoiceBridge is expected to run on a range of consumer-grade devices (desktops, laptops, tablets) with standard microphones and evolving OS audio stacks. Variations in microphone quality, form factor, and host processing capabilities, as well as future improvements in consumer audio hardware, should be handled by the audio acquisition and preprocessing modules (e.g., Microphone Manager and VAD/NoiseFilter) without impacting higher layers.

**AC2:** Speech-to-text engine and language support.

The system might expand to support additional languages and dialects beyond English requiring adaptations in ASR models and intent interpretation logic. Continuous improvements in speech recognition accuracy and noise filtering will also necessitate regular updates.

**AC3:** User profile and personalization management.

Personalization features may evolve to accommodate changing user speech patterns and preferences, demanding updates in model fine-tuning, adaptive prompting based on confidence, and profile management strategies.

**AC4:** Command mapping and browser automation protocols.

Browser capabilities, permission models, and automation interfaces (such as extension manifests and scripting APIs) are expected to evolve over time. New commands, revised command schemas, and updated browser/OS integration will require changes localized to the Command Orchestration and Browser Orchestrator modules, which are designed to isolate such API-level changes from the rest of the system.

**AC5:** User interface and accessibility compliance.

Responsive adjustments to UI designs and accessibility layers will be required to stay

compliant with evolving WCAG guidelines and to address user feedback for improved usability.

**AC6:** Error handling and recovery policies.

Classification schemes, messaging protocols, retry/backoff strategies, and compensation mechanisms may be enhanced to increase robustness and user experience quality.

**AC7:** Session lifecycle and interaction flow.

Changes may extend session duration limits, improve state persistence, and better handle asynchronous or interrupted user interactions.

**AC8:** Data privacy, audit logging, and security.

Adaptations will be necessary to comply with evolving data privacy laws and security best practices, affecting encryption, audit trail formats, and consent mechanisms.

**AC9:** Personalization and prompting enhancements.

Enhancements aiming at more context-aware and confidence-driven user assistance will continually refine prompting and instruction modules.

## 4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decision should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

**UC1:** Switching to specialized or non-consumer deployment platforms.

VoiceBridge is designed to run on consumer-grade devices using standard microphones and common operating systems. Supporting specialized assistive hardware would require major changes to hardware assumptions, certification, and system integration, and is out of scope for this project.

**UC2:** Offloading speech processing or command execution to external or cloud systems.

To protect user privacy and maintain responsiveness, VoiceBridge performs all key processing locally. Changing this would require major architecture redesign and is not planned.

**UC3:** Fundamental changes to the speech recognition approach.

The system uses speech recognition models adapted for users with speech impairments. Major changes to the recognition method—such as replacing local processing with a fully cloud-based approach—would be disruptive and are not planned.

**UC4:** Complete redesign of command mapping and browser control modules.

The system uses a browser-bridge automation pattern, with dedicated modules (Command Orchestrator, Browser Orchestrator) that encapsulate browser APIs and OS



integration. Updates to support new browser APIs or extensions are expected, but replacing the underlying automation architecture is not planned.

**UC5:** Abandoning modular design for a monolithic or radically different architecture.

Modularity was chosen for maintainability and scalability. Changing to a less modular approach would increase complexity and reduce flexibility.

## 5 Module Hierarchy

This section provides an overview of the module design. The system is organized into architectural layers, each containing modules that encapsulate specific responsibilities. Table 1 presents the hierarchy of modules by their layers, including their primary purposes and responsibilities. The modules listed below, which are leaves in the hierarchy tree, are the modules that will be implemented.

### 5.1 Architectural Terminology

To clarify the structural organization of this system, we distinguish between three key architectural concepts:

**Layer:** A layer is a horizontal grouping of modules that share a common level of abstraction and responsibility within the system architecture. Layers enforce separation of concerns by restricting dependencies—modules within a layer may only depend on modules in lower layers, never on those in higher layers. For example, the Presentation Layer contains all user-facing components (M1-M3), while the Data Management Layer (M9-M14) handles persistence and security. Layers represent what broad categories of functionality exist.

**Module:** A module is a self-contained software component that encapsulates a specific set of secrets (design decisions) and provides a well-defined set of services through its interface. Each module is the fundamental unit of implementation and change—modifications to a module’s internal implementation should not require changes to other modules, provided its interface remains stable. Modules represent how specific responsibilities are implemented and hidden behind abstractions.

**Orchestrator:** An orchestrator is a specialized type of module whose primary responsibility is coordinating the interactions between other modules to accomplish complex workflows. Unlike standard modules that focus on a single domain-specific task, orchestrators manage sequencing, decision-making, and data flow across module boundaries. In this system, M5 (Command Orchestrator) exemplifies this pattern by synthesizing speech transcripts into browser commands through coordination with M4, M16 (Prompting Module), M7 (Browser Orchestrator), and M8 (Session Manager). Orchestrators represent when modules interact and why certain sequences occur.

## Module Classification

Modules are classified as either user-facing or internal, based on their role in the system. User-facing modules (M1: User Interface, M2: Accessibility Layer, M3: Feedback Display) directly interact with the user and must meet accessibility and availability requirements. All other modules are internal; their focus is on correctness, maintainability, and security.

## 5.2 Design Rationale

**Separation of Concerns:** The architecture strictly separates presentation (M1-M3), processing (M4-M6), control (M7-M8), data (M9-M14), input (M15-M16), and personalization (M16-M17). This ensures that changes in one layer (e.g., switching STT engines) do not cascade across unrelated modules.

**Security by Design:** Security-critical functions are isolated in dedicated modules (M11-M14) with restricted access patterns. Credentials, encryption keys, and audit logs are managed independently to support defense-in-depth.

**Accessibility First:** The Accessibility Layer (M2) is a first-class architectural component rather than an afterthought, ensuring WCAG compliance is enforced systematically across all UI components.

**LLM-Driven Command Synthesis:** Unlike traditional intent-based systems with fixed command registries, the Command Orchestrator (M5) uses prompt engineering and schema validation to synthesize commands dynamically, enabling flexible natural language understanding without hardcoded mappings.

**Personalization Without Privacy Trade-offs:** The Model Tuner (M17) and User Profile Manager (M10) work in tandem with Encryption (M13) to enable user-specific adaptations while maintaining data anonymization for model training.

## 5.3 Module Listing

The following 18 modules represent the leaf nodes in the architectural decomposition and will be implemented as distinct software components:

- **User Interface Module** - Manages visual presentation and user interaction through browser DOM elements
- **Accessibility Layer** - Ensures WCAG compliance and provides accessible interfaces for users with disabilities
- **Feedback Display Module** - Manages display and lifecycle of user-facing feedback messages and notifications
- **Speech-to-Text Engine** - Converts audio input into accurate text transcripts using acoustic and language models

- **Command Orchestrator** - Synthesizes natural language into validated, executable browser commands using LLM-based reasoning
- **Error Feedback** - Normalizes errors into user-friendly messages with recovery options and logging capabilities
- **Browser Controller** - Manages communication with browser automation layer through transport protocols and session handling
- **Session Manager** - Manages session lifecycle, state persistence, and expiry handling for user interactions
- **Data Management Layer** - Handles secure storage, retrieval, backup, and retention of application data and user files
- **User Profile Manager** - Manages user profiles, preferences, and consent records with secure identification
- **Audit Logger** - Provides tamper-proof logging, querying, and anomaly detection for system activities
- **Credential Manager** - Handles authentication, session token issuance, and secure password storage
- **Encryption Manager** - Manages encryption, key rotation, data anonymization, and integrity verification
- **Out-of-Scope Handler** - Validates command safety, provides fallback handling, and reports unsafe operations
- **Audio Capture** - Captures and normalizes audio input from system microphone using voice activity detection (VAD) on audio stream
- **Prompting Module** - Generates contextual, accessible, and confidence-based prompts for user clarification
- **Model Tuner** - Incrementally fine-tunes Whisper-small via LoRA adapters on new user audio, with experience replay to prevent catastrophic forgetting

## 6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the (SRS). In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 1.

| <b>Architectural Layer</b>            | <b>Included Modules</b>                                                 | <b>Purpose / Responsibilities</b>                                                                                                                                                                                                        |
|---------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Presentation Layer</b>             | User Interface Module<br>Accessibility Layer<br>Feedback Display Module | Manages direct user interaction and feedback presentation. Displays transcribed text, confirmations, feedback messages, and status indicators. Ensures compliance with WCAG accessibility standards including contrast, font, and color. |
| <b>Control (Orchestration) Layer</b>  | Browser Orchestrator<br>Session Manager                                 | Coordinates and manages flow between modules. Tracks session states such as capture, transcribe, confirm, and execute. Manages events, exceptions, retries, and cancellations to ensure smooth operation.                                |
| <b>Application (Processing) Layer</b> | Error Feedback Module<br>Command Orchestrator<br>Speech-to-Text Engine  | Performs speech recognition and intent processing. Converts speech audio to text, interprets user intent, maps intent to browser or OS commands, executes commands, and handles error feedback.                                          |
| <b>Data Management Layer</b>          | Data Storage Manager<br>User Profile Manager<br>Audit Logger            | Maintains persistent data and user personalization. Stores user speech samples, preferences, command mappings, transcripts, and logs. Supports diagnostics and evaluation.                                                               |
| <b>Security Layer</b>                 | Credential Manager<br>Encryption Manager<br>Out-of-Scope Handler        | Manages authentication, encryption, and system boundaries. Ensures secure integration with external APIs and operating system interfaces. Handles login confirmations.                                                                   |
| <b>Input Processing Layer</b>         | Microphone Manager<br>VAD and Noise Filter                              | Controls microphone hardware and audio capture. Filters noise to improve voice input quality.                                                                                                                                            |
| <b>Personalization Layer</b>          | Prompting Module<br>Shortcut Manager<br>Model Tuner                     | Provides user prompting, model fine-tuning, and creation of personalized shortcuts. Supports optional personalization based on confidence scores and usage history.                                                                      |

Table 1: Module Hierarchy

## 7 Module Decomposition

### Presentation Layer Modules

#### M1: User Interface Module

##### M1 Secrets:

- Representation of UI state, layout, and visible components
- Mapping of raw browser events into UiEvents
- Management of keyboard and screen-reader focus

##### Services:

- Initialize and render UI
- Receive and normalize input events into UiEvents
- Route feedback items
- Update keyboard and screen-reader focus

**Implemented By:** `UserInterface` (`VoiceBridge`) module using the browser rendering engine and DOM/event APIs.

#### M2: Accessibility Layer

##### M2 Secrets:

- Encoding of accessibility configuration (`AccessConfig`)
- ARIA attribute mappings
- Validation rules for UI elements against WCAG standards

##### Services:

- Apply accessibility options
- Announce messages
- Check UI elements and report problems using `ValidationReports`

**Implemented By:** `AccessibilityLayer` (`VoiceBridge`) module operating on UI elements exposed by `UserInterface`.

### **M3: Feedback Display Module**

#### **M3 Secrets:**

- Internal representation of FeedbackItems keyed by UUID
- Policies for creation, clearance, and recovery options of feedback messages

#### **Services:**

- Attach to parent UI
- Show messages of given types
- Clear messages
- Create recovery options for feedback items

**Implemented By:** FeedbackDisplay (VoiceBridge) module rendering via `UIKit` and coordinating with `AccessibilityLayer`.

### **Application (Processing) Modules**

#### **M4: Speech-to-Text Engine**

#### **M4 Secrets:**

- Configuration (`AsrConfig`) and internal processing details of the speech-to-text engine
- Methods for converting audio input into accurate text transcripts

#### **Services:**

- Initialize engine
- Process audio to produce transcripts with confidence metadata
- Reset internal state

**Implemented By:** `SpeechToTextEngine` (VoiceBridge) module wrapping acoustic and language models.

## M5: Command Orchestrator

M5

### Secrets:

- Prompt engineering strategies and system instructions for command clarification and synthesis.
- Session-based conversation context and short-term memory management for multi-turn interactions.
- LLM API integration details (e.g., Gemini API configuration, dynamic context injection).
- Heuristics for mapping natural language to browser-specific schemas without a fixed registry.
- Safety guardrail heuristics for blocking malicious or ambiguous commands while allowing informational queries.

### Services:

- Initialize the orchestrator with system prompts, API key, model configuration, and optional custom system prompt.
- Process user input to determine if intent is clear or needs clarification, returning either a clarified browser command or a clarification question.
- Apply safety guardrails to prevent execution of harmful or ambiguous commands.
- Reset session context and clear conversation history.
- Accept supplemental user input when the recognized intent is incomplete, process it with the original transcript and session context, and generate an updated command.

**Implemented By:** CommandOrchestrator (VoiceBridge) module using an external LLM API, dynamic context injection, and a JSON schema validation layer.

## M6: Error Feedback

M6

### Secrets:

- Expected error types
- Message templates and localization hooks
- Mapping of internal/domain errors to user-facing message keys and recovery options

- Redaction rules for logs and UI

**Services:**

- Display error notifications
- Display recovery prompts with options
- Dismiss items
- Log structured error events
- Normalize and map errors to UI messages and recovery actions

**Implemented By:** UI Notification Layer (VoiceBridge) and Shared Error Handling Module (M11).

## Control (Orchestration) Modules

### M7: Browser Orchestrator

**M7 Secrets:**

- Construction of the `browser_use` agent (`Agent`) and LLM wrapper (`ChatBrowserUse`)
- How the natural-language `command` is passed to the agent (`task=command`)
- Interpretation/formatting of the returned execution `history`

**Services:**

- Execute a natural-language browser command using an active `Browser` instance

**Implemented By:** `src/control/browser_orchestrator.py` (VoiceBridge).

### M8: Session Manager

**M8 Secrets:**

- Management of a single shared browser instance (`_browser`) for the whole application
- Browser start/stop lifecycle and cleanup logic
- Keep-alive configuration for reusing the same browser across multiple commands (`keep_alive=True`)

**Services:**

- Start the shared browser session
- Stop the shared browser session
- Provide access to the active `Browser` instance (or `None` if not running)

**Implemented By:** `src/control/session_manager.py` (VoiceBridge).



## Data Management Modules

### M9: Storage Management Module

#### M9 Secrets:

- Database connection credentials
- Cloud API tokens
- Encryption keys

#### Services:

- `DataStorageService`: Stores and retrieves transcripts, user data, and configuration files.
- `BackupService`: Performs scheduled or manual backups to secure cloud storage.
- `DataRetrievalService`: Provides indexed access to stored files for authorized modules.
- `DataRetentionService`: Enforces deletion and retention rules based on time or policy.

**Implemented By:** `DataStorageManager` (`VoiceBridge`).

### M10: User Profile Management Module

#### M10 Secrets:

- User tokens and refresh keys
- Profile encryption keys
- Hashed user identifiers

#### Services:

- `ProfileCreationService`: Initializes new user profiles with unique IDs.
- `PreferenceService`: Saves and retrieves user preferences and personalization data.
- `ConsentManagementService`: Records and verifies consent for data collection and personalization.

**Implemented By:** `UserProfileManagerImpl` (`VoiceBridge`).

## **M11: Audit Logging Module**

### **M11 Secrets:**

- Log signing key
- Audit database token
- Log encryption key

### **Services:**

- ActivityLogService: Logs system and user activities for traceability.
- LogQueryService: Provides authorized access to query and review logs.
- AnomalyDetectionService: Detects suspicious activity and security anomalies.

**Implemented By:** AuditLoggerImpl (VoiceBridge).

## **M12: Credential Management Module**

### **M12 Secrets:**

- Vault root key
- Token signing keypair
- OAuth client secrets

### **Services:**

- AuthenticationService: Validates user credentials during login.
- SessionTokenService: Issues and validates secure access tokens (JWT/OAuth).
- PasswordVaultService: Stores and retrieves hashed passwords securely.

**Implemented By:** CredentialManagerImpl (VoiceBridge).

## **M13: Encryption Management Module**

### **M13 Secrets:**

- Private encryption keypair
- TLS certificates for secure communication
- Key rotation schedule metadata

### **Services:**

- DataEncryptionService: Encrypts data at rest and in transit.
- KeyManagementService: Generates, rotates, and revokes cryptographic keys.
- AnonymizationService: Removes identifiable data prior to model training or storage.
- IntegrityVerificationService: Ensures stored or transmitted data has not been altered.

**Implemented By:** EncryptionManagerImpl (VoiceBridge).

## **M14: Out-of-Scope Handling Module**

### **M14 Secrets:**

- Command allowlist/denylist configuration
- Safety policy file checksum
- Secure incident reporting token

### **Services:**

- CommandValidationService: Validates command scope and prevents unsafe actions.
- UserSafetyService: Provides safe fallback messages and cancel options.
- RecoveryService: Rolls back partially executed or invalid operations.
- IncidentReportingService: Reports anomalies and unsafe commands to the AuditLogger.

**Implemented By:** OutOfScopeHandlerImpl (VoiceBridge).

## Input Processing Layer Modules

### M15: Real-time Audio Capture with Voice Activity Detection

M15

#### Secrets:

- Audio device configuration and sampling parameters (sample rate, channels, chunk size) encapsulated in `AudioConfig`
- Hybrid speech detection combining WebRTC VAD (aggressiveness 0–3) with an optional RMS energy threshold
- Six-state capture machine (IDLE, LISTENING, SPEECH\_DETECTED, RECORDING, PROCESSING, STOPPED) and its thread-safety strategy
- Pre-roll ring buffer used to recover audio onset that precedes speech detection
- Silence-run and max-duration counters that trigger automatic recording termination
- Audio format pipeline: float32 stream input → int16 bytes for VAD → normalized float32 output for Whisper

#### Services:

- Validate and initialize audio capture parameters via `AudioConfig`
- Start and stop the continuous microphone stream (`start`, `stop`)
- Automatically detect speech onset and record complete utterances, emitting each as a float32 array with metadata (duration, sample count, timestamp) via a caller-supplied callback
- Enumerate available audio input devices (`list_audio_devices`)
- Expose runtime diagnostics: total recordings, captured audio seconds, VAD errors, and stream errors (`get_stats`)

**Implemented By:** `audio_capture.py` (VoiceBridge) (`AudioConfig`, `AudioState`, `RealtimeAudioCap`) using the `sounddevice` and `webrtcvad` libraries.

## Personalization Layer

### M16: Prompting Module

M16 Secrets:

- Prompt templates and personalization rules

- Condition-based variation (history, context)
- Confidence-based prompt selection logic

**Services:**

- Generate user prompts
- Provide clarifying follow-ups when intent confidence is low
- Produce culturally inclusive and accessibility-enabled copy

**Implemented By:** Application prompt engine (VoiceBridge).

## M17: Model Tuner

**M17 Secrets:**

- Incremental fine-tuning strategy: only LoRA adapter weights (targeting Whisper-small decoder `q_proj/v_proj` layers) are trained; the base model is frozen
- Seen-files ledger (`seen_files.json`) that tracks which audio samples have already been incorporated, ensuring each recording is trained on exactly once
- Experience-replay policy: a random subset of previously-seen samples (sized `replay_ratio`  $\times$  `#new` files) is mixed into each run to prevent catastrophic forgetting
- LoRA hyperparameters (rank, scaling factor, dropout) and their effect on adapter capacity vs. training speed
- Adapter persistence path (`models/adapters`) and the in-place overwrite protocol used to keep a single live adapter up to date across runs

**Services:**

- Prepare a training run: filter metadata to unseen files, load or initialise the LoRA adapter, and build train/test `DataLoaders` (`setup`)
- Train the adapter on new audio data for a configurable number of epochs, returning per-epoch losses (`train`)
- Evaluate adapter quality on the held-out test split, returning average cross-entropy loss (`evaluate`)
- Persist the updated adapter to disk and advance the seen-files ledger (`save`)
- Expose whether unseen training data is available for the current run (`has_new_data`)

**Implemented By:** `whisper_lora_trainer.py` (VoiceBridge) (`WhisperLoRATrainer`) using HuggingFace `transformers`, `peft` (LoRA), and PyTorch.

## M18: Shortcut Manager

### M18 Secrets:

- Internal mapping of shortcut IDs to recorded command sequences
- Temporary in-memory recording buffer for active shortcut creation
- Recording state indicating whether shortcut capture is currently active
- Persistent storage path and serialization format for saved shortcuts

### Services:

- Start and stop recording of multi-step browser command shortcuts
- Store and persist recorded shortcuts for future reuse
- Return the current set of saved shortcuts
- Delete an existing shortcut by ID
- Report whether shortcut recording is currently active
- Cancel an in-progress recording without saving

**Implemented By:** `ShortcutManager` (`VoiceBridge`) class in the control/runtime layer with JSON-backed file persistence.

## 8 Traceability Matrix

The traceability matrices show how each requirement is implemented by specific modules and how each anticipated change is isolated within a single module.

| Req. | Modules         |
|------|-----------------|
| FR1  | M15, M4, M8     |
| FR2  | M4, M17, M9     |
| FR3  | M1, M2, M3, M6  |
| FR4  | M5, M6          |
| FR5  | M7, M7, M11, M6 |

Table 2: Trace Between Functional Requirements and Modules

| Req. | Modules         |
|------|-----------------|
| LF1  | M1, M2, M3, M6  |
| LF2  | M1, M2, M16, M3 |

Table 3: Trace Between Look & Feel Requirements and Modules

| Req. | Modules         |
|------|-----------------|
| UH1  | M1, M2, M3, M6  |
| UH2  | M10, M9, M5, M6 |
| UH3  | M1, M2, M16, M8 |
| UH4  | M6, M3, M2, M16 |
| UH5  | M2, M1, M3, M6  |

Table 4: Trace Between Usability & Humanity Requirements and Modules

| Req. | Modules             |
|------|---------------------|
| PF1  | M4, M5, M6, M7, M7  |
| PF2  | M5, M6, M7, M6, M11 |
| PF3  | M4, M17, M5         |
| PF4  | M11, M8, M6, M7     |
| PF5  | M8, M9, M10         |
| PF6  | M8, M9, M7          |
| PF7  | M9, M10, M11, M12   |

Table 5: Trace Between Performance Requirements and Modules

| Req. | Modules               |
|------|-----------------------|
| OER1 | M15, M4, M11          |
| OER2 | M1, M2, M7, M8        |
| OER3 | M5, M6, M7            |
| OER4 | M9, M10, M11, M12, M8 |
| OER5 | M8, M9, M11, M12      |

Table 6: Trace Between Operational & Environmental Requirements and Modules

| Req. | Modules               |
|------|-----------------------|
| MS1  | M8, M9, M10, M11, M12 |
| MS2  | M1, M2, M3, M6, M16   |
| MS3  | M1, M2, M7, M8, M11   |

Table 7: Trace Between Maintainability & Support Requirements and Modules

| Req. | Modules                |
|------|------------------------|
| SEC1 | M12, M13, M10, M9, M8  |
| SEC2 | M5, M6, M7, M6, M11    |
| SEC3 | M9, M10, M11, M12, M13 |
| SEC4 | M11, M9, M8, M12       |
| SEC5 | M11, M6, M7, M7        |

Table 8: Trace Between Security Requirements and Modules

| Req. | Modules             |
|------|---------------------|
| CUL1 | M2, M3, M6, M16, M5 |

Table 9: Trace Between Cultural Requirements and Modules

| Req. | Modules                |
|------|------------------------|
| CPL1 | M9, M10, M11, M12, M13 |
| CPL2 | M1, M2, M3, M6         |

Table 10: Trace Between Compliance Requirements and Modules

## 9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. Parnas (1978) said of two programs A and B that A *uses* B if correct execution of B may be necessary for A to complete the task described in its specification. That is, A *uses* B if there exist situations in which the correct functioning of A depends upon the availability of a correct implementation of B. Figure 1 illustrates the use relation between the modules. It can be seen that the graph is a directed acyclic graph (DAG). Each level of the hierarchy offers a testable and usable



subset of the system, and modules in the higher level of the hierarchy are essentially simpler because they use modules from the lower levels.

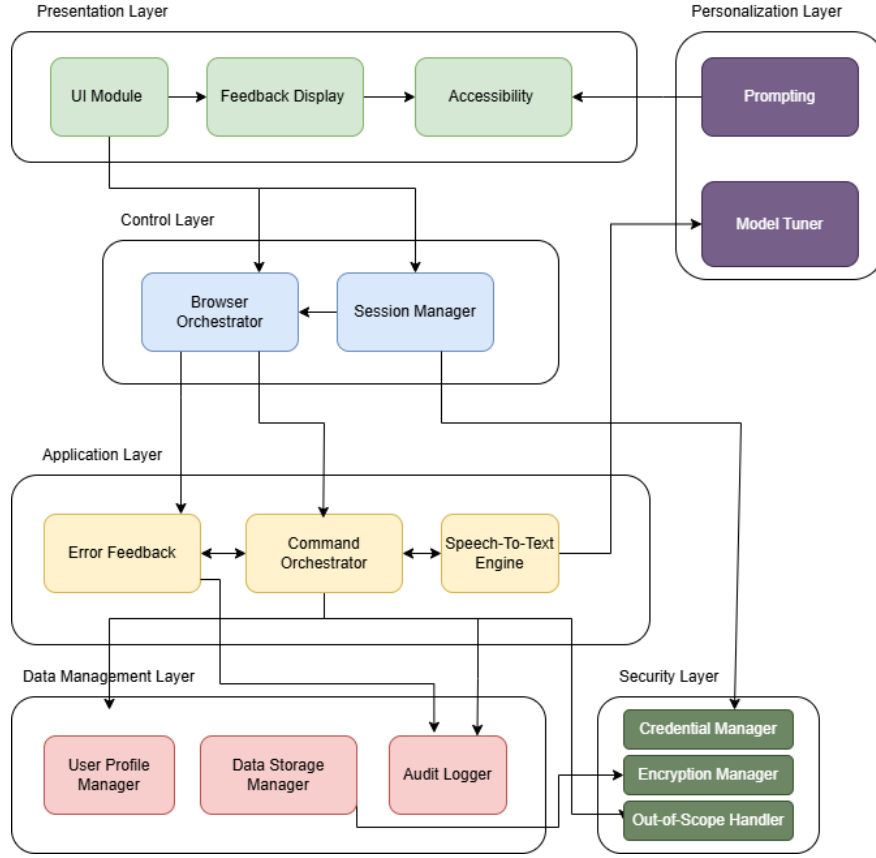


Figure 1: Use hierarchy among modules

## 10 User Interfaces

This section presents the primary user interface components of the VoiceBridge system. The interfaces are designed with accessibility, clarity, and low cognitive load in mind.



Figure 2: VoiceBridge browser toolbar icon

The browser toolbar icon displayed in Figure 2 serves as the primary entry point for VoiceBridge within the browser environment.

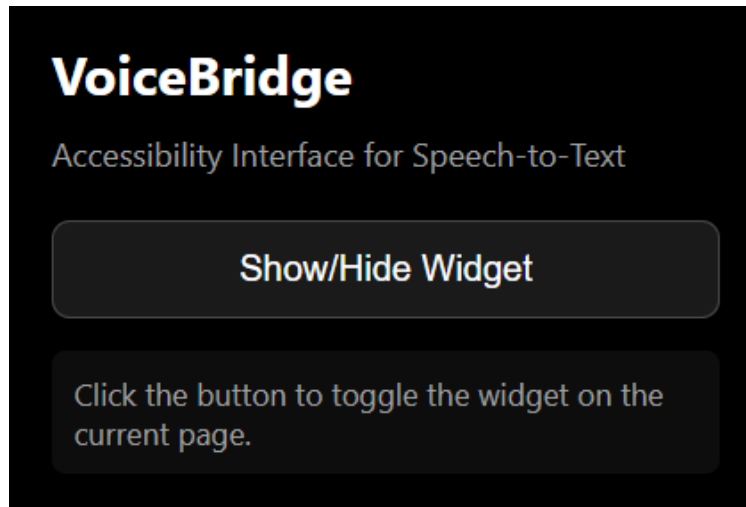


Figure 3: Browser extension widget toggle interface

The browser extension control panel allows users to enable or disable the VoiceBridge widget on the current webpage using a single-action toggle.

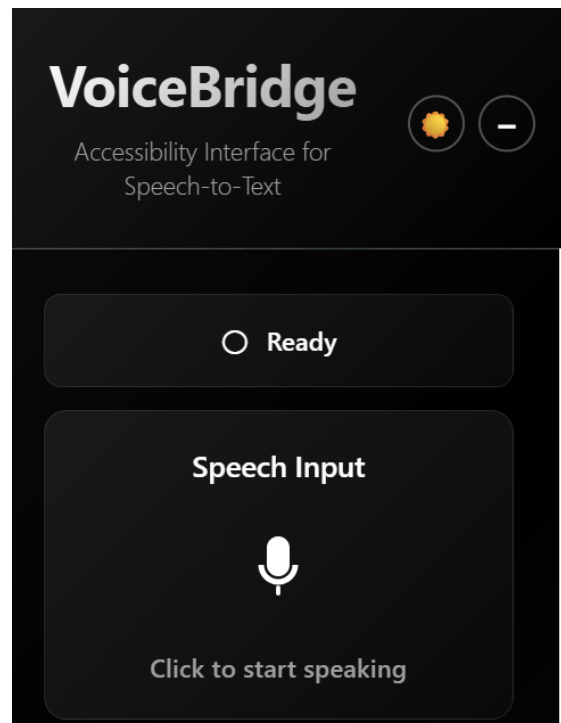


Figure 4: Main VoiceBridge application interface

The main application interface shown in Figure 4 integrates system status, speech input, and transcription components into a single cohesive layout.

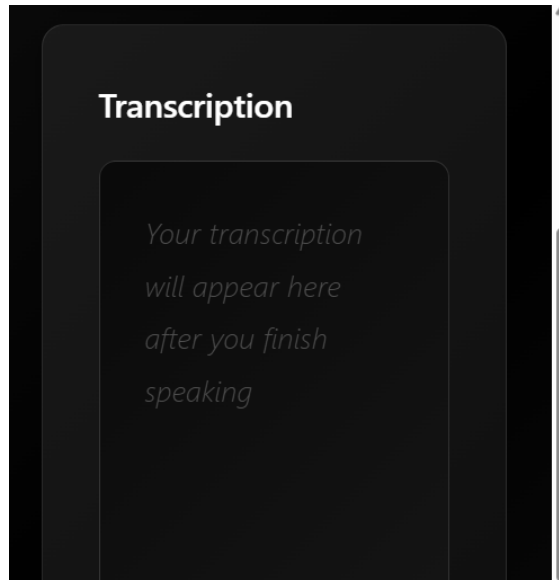


Figure 5: Live transcription output panel

Figure 5 presents the transcription panel where speech is displayed after the user finishes speaking. This component allows users to visually verify the system’s interpretation of their speech before any further action is taken.

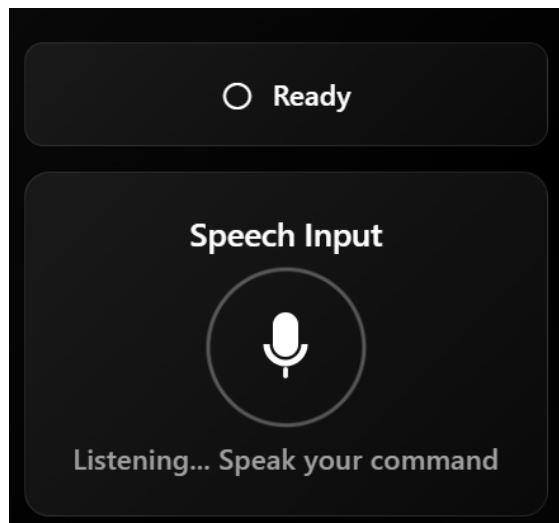


Figure 6: Speech input interface while listening

The speech input interface, shown in Figure 6, provides clear feedback on the system’s listening state. A large microphone icon indicates when speech input is active, while supporting text guides the user to speak their command.

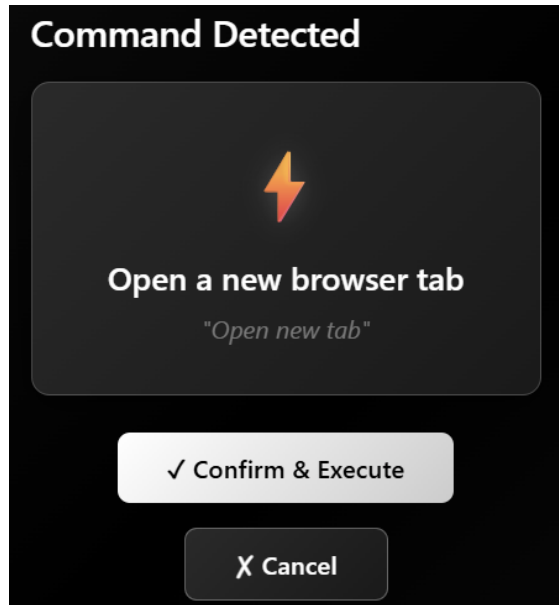


Figure 7: Detected voice command confirmation screen

Figure 7 shows the command detection interface that appears after a voice command has been successfully recognized. The detected intent (e.g., opening a new browser tab) is displayed, along with the interpreted spoken phrase.

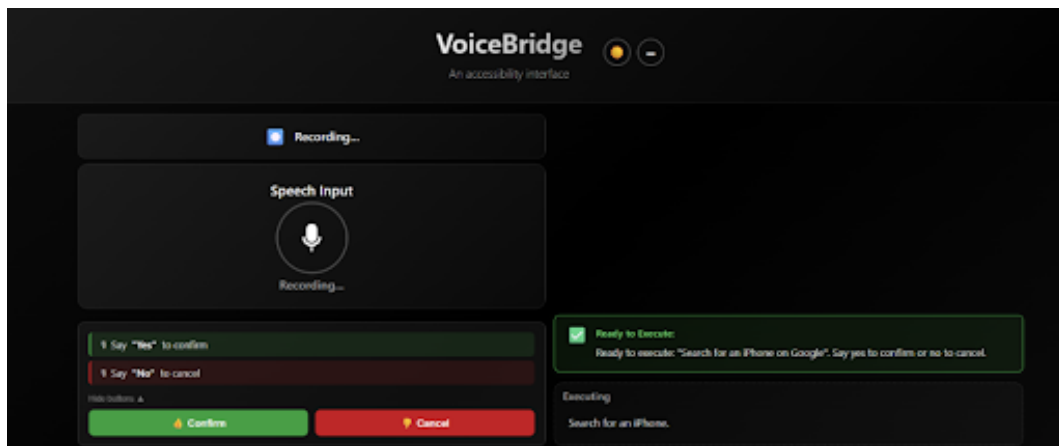


Figure 8: Confirmation screen with command refinement input

Figure 8 shows the confirmation screen that verifies recognized commands by recording the user’s spoken response or allowing them to click Confirm or Cancel buttons, enabling refinement of incomplete intents like “search for an iPhone on Google” before execution.

## 11 Design of Communication Protocols

VoiceBridge uses typed procedure calls between modules following the [MIS](#) details. All communication is synchronous with structured data types and exception handling.

- **Typed Access Programs:** Module interfaces are defined in the [MIS](#), utilizing request-response patterns for inter-module communication.
- **Structured Messages:** Data types such as `UiEvent`, `Transcript`, `Command`, and `FeedbackItem` are used to standardize communication.
- **Exception Handling:** Typed errors (e.g., `TimeoutError`, `StoreError`) are propagated and converted into user feedback where applicable.
- **Session Tokens:** UUID-based authentication is managed via the [M12](#) Credential Manager to ensure secure communication.
- **Data Encryption:** The [M13](#) Encryption Manager handles encryption at storage and communication boundaries to maintain data security.

## 12 Timeline

Core functionality such as speech-to-text and command orchestrator was developed collectively. Each team member took ownership of specific modules to ensure accountability and progress. The schedule in [Table 11](#) reflects the planned completion dates for each module, along with the responsible team member.

## References

- David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.
- David L. Parnas. Designing software for ease of extension and contraction. In *ICSE '78: Proceedings of the 3rd international conference on Software engineering*, pages 264–277, Piscataway, NJ, USA, 1978. IEEE Press. ISBN none.
- D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.

| <b>Module Name</b>      | <b>Completion Date</b> | <b>Responsible Member</b> |
|-------------------------|------------------------|---------------------------|
| User Interface Module   | January 15, 2026       | Kelvin                    |
| Speech-to-Text Engine   | January 15, 2026       | Luna                      |
| Command Orchestrator    | January 15, 2026       | Rawan                     |
| browser orchestrator    | January 15, 2026       | Mazen                     |
| Accessibility Layer     | January 18, 2026       | Mazen                     |
| Feedback Display Module | January 18, 2026       | Rawan                     |
| Error Feedback          | January 18, 2026       | Kelvin                    |
| Session Manager         | January 21, 2026       | Luna                      |
| Prompting Module        | January 21, 2026       | Rawan                     |
| User Profile Manager    | January 21, 2026       | Kelvin                    |
| Data Management Layer   | January 24, 2026       | Rawan                     |
| Audit Logger            | January 24, 2026       | Mazen                     |
| Credential Manager      | January 24, 2026       | Luna                      |
| Encryption Manager      | January 27, 2026       | Rawan                     |
| Out-of-Scope Handler    | January 27, 2026       | Kelvin                    |
| Microphone Manager      | January 27, 2026       | Mazen                     |
| VAD Noise Filter        | January 27, 2026       | Luna                      |
| Model Tuner             | January 30, 2026       | Kelvin                    |

Table 11: Module Completion Schedule